

WEB APPLICATION SECURITY ASSESSMENT

OWASP JUICE SHOP

Kali Linux • Docker • Burp Suite • Firefox

Assessment Type	Authorized local security-testing laboratory
Target	OWASP Juice Shop – http://localhost:3000
Primary Tools	Burp Proxy, Repeater, Intruder, Docker, Firefox
Evidence	Project screenshots supplied for this assessment

Practical Security Testing, Evidence Analysis,
Risk Assessment and Remediation

Report Map

Each section combines detailed explanation, practical security analysis, evidence where relevant, and a reviewer-focused takeaway.

SECTION	FOCUS
Executive Summary	This project demonstrates a controlled web application security assessment of the OWASP Juice Shop running locally ...
Project Objectives	The project has five principal objectives
Scope and Ethical Boundaries	The assessed target shown in the evidence is the OWASP Juice Shop instance hosted at http://localhost:3000
Technology Stack and Laboratory Architecture	The laboratory consists of four functional layers
Docker Deployment	The deployment evidence shows the Juice Shop image being pulled from Docker Hub and then started as a named contain...
Verifying the Application	After deployment, the Juice Shop web interface is opened in Firefox
Burp Suite Proxy Configuration	The browser connection settings shown in the evidence use manual proxy configuration with HTTP proxy 127
Interception Workflow	The Proxy screenshots show interception enabled and a GET request to the local target appearing in Burp
Authentication Request Analysis	The Repeater evidence contains a POST request to <code>/rest/user/login</code> with a JSON body containing an email and password
Credential Testing with Burp Intruder	The Intruder screenshot shows a POST login request with a selected password parameter and a small simple-list paylo...
Authentication Failure Analysis	A separate Repeater screenshot shows a login attempt returning HTTP 401 Unauthorized with the message 'Invalid emai...
Authentication Input Manipulation	One Repeater screenshot shows an authentication body containing a quote-based SQL-injection-style expression in the...
Token and Session Handling	The successful login responses contain long token strings in JSON
Basket API Authorization Testing	The Repeater screenshots show an authenticated request to a basket endpoint such as <code>GET /api/BasketItems/2</code> followed...
HTTP Method and API Behavior	The basket update evidence illustrates how the same API resource can support different HTTP methods
User Endpoint and Data Exposure	The screenshot showing <code>GET /rest/user/whoami?fields=email</code> demonstrates an API endpoint that returns information abo...
Cross-Site Scripting Test	The final browser screenshot shows a search query containing an <code>iframe/javascript</code> -style XSS test string
CSRF Considerations	The project evidence includes authentication and state-changing API requests, making cross-site request forgery an ...
Findings and Risk Assessment	Based on the screenshots, the most significant areas for further investigation are authentication input handling, a...
Remediation Recommendations	1
Test Methodology and Reproducibility	The assessment follows a repeatable sequence: deploy the target, verify availability, configure the browser proxy, ...
Limitations	The report is based on the supplied screenshots rather than a live re-execution of the environment
Conclusion	This project demonstrates a complete introductory web application security testing workflow against an intentionall...
Appendix A – Evidence Index	Figure 1: Burp login request and authentication response

Appendix B – Security Testing Checklist	Before testing: confirm authorization, define scope, verify the local target, and create a clean test account
Appendix C – Final Project Summary	The completed laboratory can be summarized as: deploy Juice Shop in Docker; access it on localhost:3000; route brow...

Executive Summary

SECTION 01 | SECURITY ANALYSIS

This project demonstrates a controlled web application security assessment of the OWASP Juice Shop running locally in a Docker container. The testing workflow combines Kali Linux, Docker, Firefox, Burp Suite Proxy, Repeater, and Intruder. The evidence shows the application being deployed on localhost:3000, browser traffic being routed through Burp Suite on 127.0.0.1:8080, HTTP requests being intercepted and replayed, authentication responses being examined, a password-test set being exercised with Intruder, an authenticated basket API request being manipulated, a user endpoint being queried, and an XSS-oriented search input being tested.

The objective is not to attack a production system. Juice Shop is an intentionally vulnerable training application, making it suitable for learning how application-layer vulnerabilities appear in real HTTP traffic. The screenshots are treated as primary evidence. Where a screenshot does not conclusively prove exploitation, the report describes the activity as a test or observation rather than claiming a confirmed vulnerability.

ASSESSMENT & SECURITY TAKEAWAY

Security perspective: the evidence demonstrates the complete movement from application deployment to HTTP-level testing. The strongest observations are the successful authentication responses, distinguishable failed-login response, authenticated bearer-token requests, and API operations against basket resources. These observations provide a practical basis for discussing authentication, authorization, session management, input validation, and API security. The report deliberately separates evidence from assumptions so reviewers can identify which conclusions are confirmed by screenshots and which require additional server-side verification.

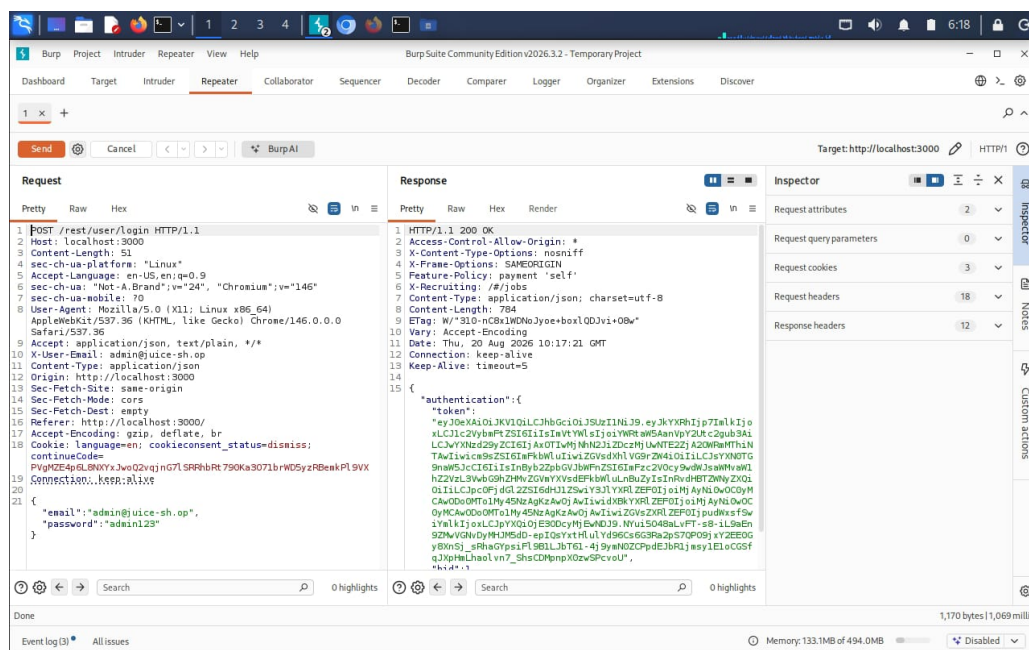


Figure 1: Burp Repeater: login request and successful JSON authentication response.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Project Objectives

SECTION 02 | SECURITY ANALYSIS

The project has five principal objectives. First, establish a reproducible local vulnerable-web-app laboratory using Docker. Second, configure the browser to send HTTP/HTTPS traffic through Burp Suite so requests can be inspected. Third, understand the structure of authentication requests, including JSON bodies, cookies, authorization headers, and token-bearing responses. Fourth, use Burp Repeater and Intruder to perform controlled security tests against the local target. Fifth, document the security implications and recommend defensive controls such as strong authentication, rate limiting, authorization checks, input validation, output encoding, CSRF protections, and secure token handling.

A secondary objective is to develop an evidence-driven methodology. Each test begins with a normal request, identifies the parameter or endpoint under examination, changes only the relevant input, observes the HTTP response, and records the result. This approach reduces accidental changes and makes the assessment easier to reproduce.

ASSESSMENT & SECURITY TAKEAWAY

Expected learning outcome: the student should be able to explain browser-to-application communication, identify the security importance of HTTP methods and headers, recognize authentication tokens, and understand why authorization must be checked independently for every resource. The project also demonstrates professional reporting: a security test is most useful when another tester can reproduce the request, observe the response, and understand why the result matters.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Scope and Ethical Boundaries

SECTION 03 | SECURITY ANALYSIS

The assessed target shown in the evidence is the OWASP Juice Shop instance hosted at <http://localhost:3000>. The screenshots also show Kali Linux as the testing platform and Docker as the deployment mechanism. The assessment is therefore scoped to an intentionally vulnerable local training application. No public host, third-party account, or external production service is targeted.

Security testing must always be performed with authorization. Techniques demonstrated in this report—request interception, credential testing, parameter manipulation, authorization testing, and XSS probing—can cause harm when directed at systems without permission. In a legitimate assessment, the tester should define the target, allowed methods, test window, data-handling rules, and stop conditions before beginning. For this project, the local Juice Shop environment provides the required authorization boundary.

ASSESSMENT & SECURITY TAKEAWAY

Scope control is a core professional skill. The target in this exercise is deliberately local and is reached through `localhost:3000`. Testing should stop at the authorized application boundary. The same techniques must not be transferred to public websites, organizational systems, or accounts without explicit permission. Evidence should also be handled carefully: passwords, session tokens, cookies, and other secrets should be redacted before reports are distributed outside the lab.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Technology Stack and Laboratory Architecture

SECTION 04 | SECURITY ANALYSIS

The laboratory consists of four functional layers. The first is the operating environment, represented by Kali Linux. The second is the application runtime, represented by Docker and the Juice Shop container. The third is the testing proxy, represented by Burp Suite. The fourth is the client, represented by Firefox.

The browser sends requests to the Burp proxy at 127.0.0.1:8080. Burp then forwards approved requests to the Juice Shop target at localhost:3000. This arrangement allows the tester to pause, inspect, modify, resend, and compare HTTP messages. Docker isolates the intentionally vulnerable application from the rest of the system and provides a repeatable startup process.

Conceptually: Firefox → Burp Proxy → localhost:3000 Juice Shop → HTTP response → Burp → Firefox. Repeater and Intruder operate on copies of captured requests, enabling controlled experiments without repeatedly using the browser UI.

ASSESSMENT & SECURITY TAKEAWAY

Component responsibilities: Kali Linux provides the security-testing workstation; Docker provides an isolated and repeatable application runtime; Firefox generates real browser requests; Burp Proxy observes and controls traffic; Repeater supports precise manual modification; and Intruder supports small, repeatable payload tests. This separation allows the tester to identify whether unexpected behavior originates from the client, proxy configuration, request construction, or target application.

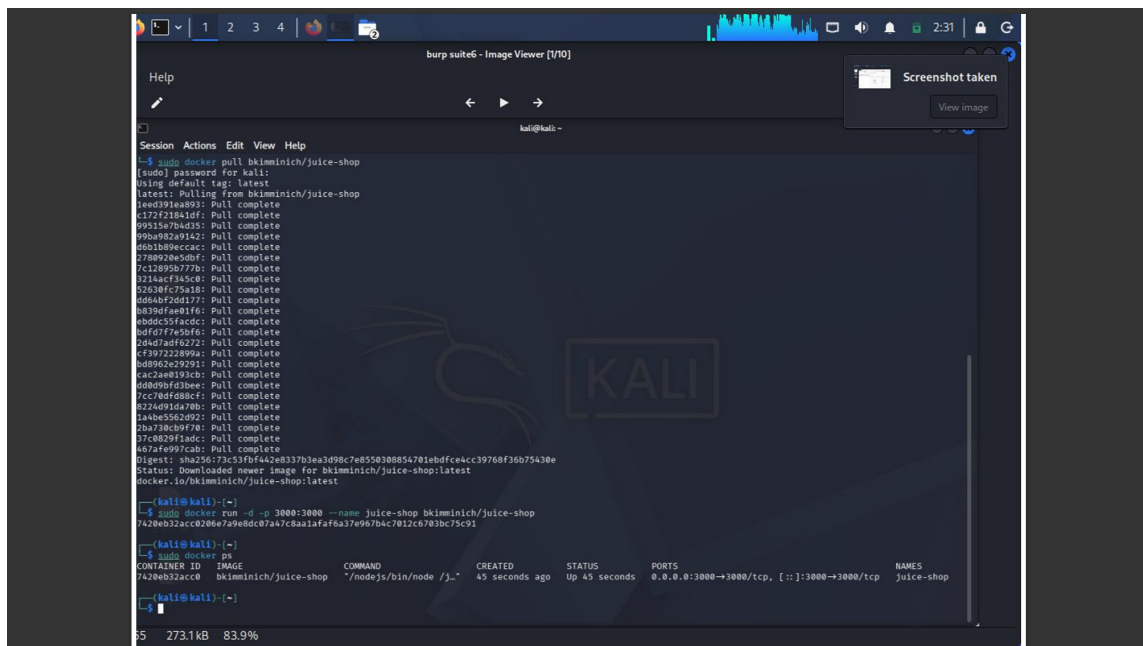


Figure 2: Docker: pulling the OWASP Juice Shop image on Kali Linux.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Docker Deployment

SECTION 05 | SECURITY ANALYSIS

The deployment evidence shows the Juice Shop image being pulled from Docker Hub and then started as a named container. The container is launched with host port 3000 mapped to container port 3000. This makes the application accessible through the local browser at <http://localhost:3000>.

Docker is valuable in a security laboratory because the vulnerable application can be started, stopped, removed, and recreated without manually installing all application dependencies. The container boundary also reduces the chance that an intentionally vulnerable application is accidentally exposed as a normal system service. For a formal laboratory, the tester should still verify network exposure, because publishing a container port on 0.0.0.0 can make it reachable from other interfaces if firewall rules permit it.

The screenshots confirm the container reached an 'Up' state and that port 3000 was published. This establishes the foundation for the remaining tests.

ASSESSMENT & SECURITY TAKEAWAY

Operational checks after container startup should include verifying that the expected container is running, confirming the published port, opening the application in a browser, and checking that the target is not unintentionally exposed beyond the intended lab network. For a clean retest, the tester can stop and recreate the container from the known image. Keeping deployment commands and image information in the project notes improves reproducibility.

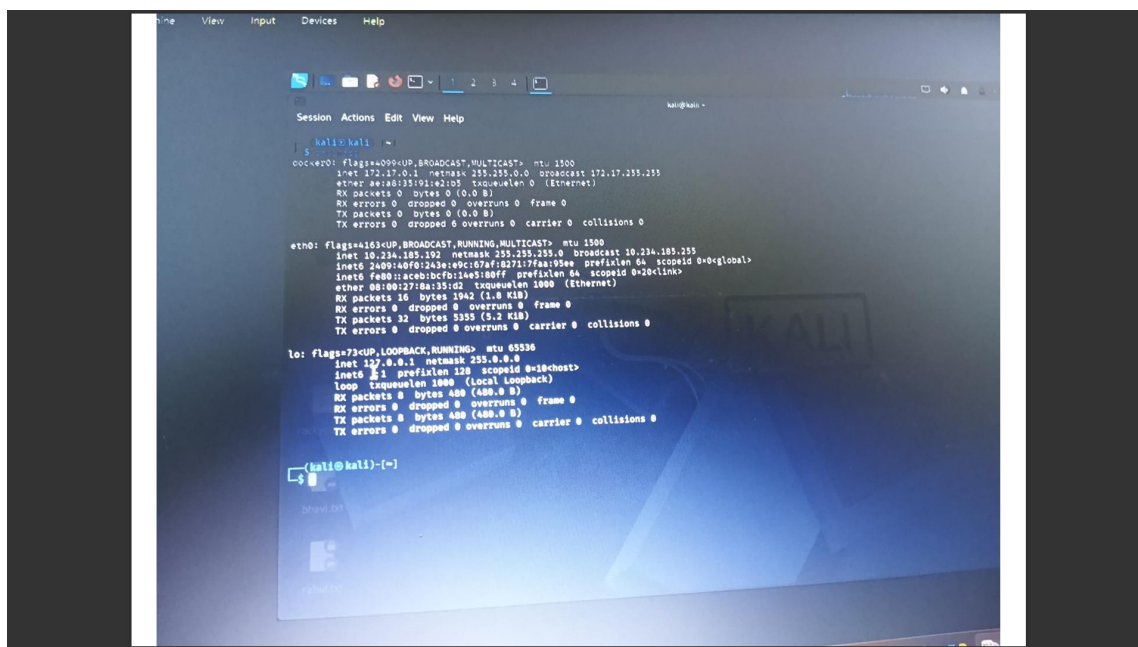


Figure 3: Docker: running the Juice Shop container with port 3000 published.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Verifying the Application

SECTION 06 | SECURITY ANALYSIS

After deployment, the Juice Shop web interface is opened in Firefox. The screenshot shows product cards, navigation controls, account and basket functions, and the application operating on localhost. This is an important baseline step: security testing should start only after the application is confirmed to be reachable and functioning normally.

A baseline request is useful because later changes in status code, response length, response body, cookies, or application behavior can be compared against the normal state. During an assessment, the tester should record the target URL, application version when known, browser configuration, proxy configuration, and any seeded test accounts. These details make the final report reproducible.

ASSESSMENT & SECURITY TAKEAWAY

Baseline verification should record normal application behavior before security manipulation. Useful baseline details include the landing page, response status, normal cookies, available account functions, and representative API requests. Later anomalies can then be compared against this baseline instead of being judged in isolation. This is particularly important for authentication, because a change from a normal 401 response to a 200 response may be significant but still requires confirmation.

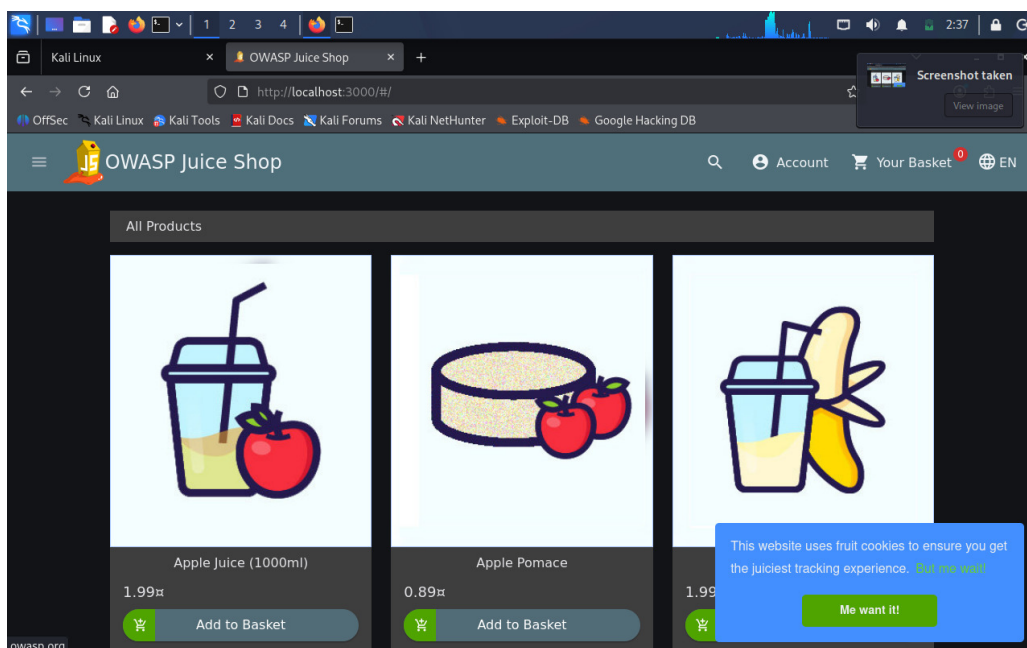


Figure 9: OWASP Juice Shop: local application running in the browser.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Burp Suite Proxy Configuration

SECTION 07 | SECURITY ANALYSIS

The browser connection settings shown in the evidence use manual proxy configuration with HTTP proxy 127.0.0.1 and port 8080, with the same proxy selected for HTTPS traffic. This directs browser traffic through Burp Suite.

The key concept is that a proxy is positioned between the client and the target. When interception is enabled, Burp can hold a request before forwarding it. The tester can inspect headers, cookies, query parameters, body parameters, and other HTTP metadata. Once the request is forwarded, the response can also be reviewed in Burp's history and related tools.

For HTTPS testing, a trusted Burp CA certificate is normally required in the browser so that Burp can inspect TLS traffic in the lab. Because the target shown is local HTTP, the screenshots primarily demonstrate the proxy routing mechanism rather than a production TLS deployment.

ASSESSMENT & SECURITY TAKEAWAY

Proxy configuration is the bridge between browser-level actions and raw HTTP analysis. A correctly configured proxy lets the tester observe methods, paths, parameters, cookies, authorization headers, and response codes without modifying the application itself. In a professional engagement, proxy scope should be restricted to the approved target and sensitive data should be protected. After the assessment, the browser should be returned to its normal proxy settings.

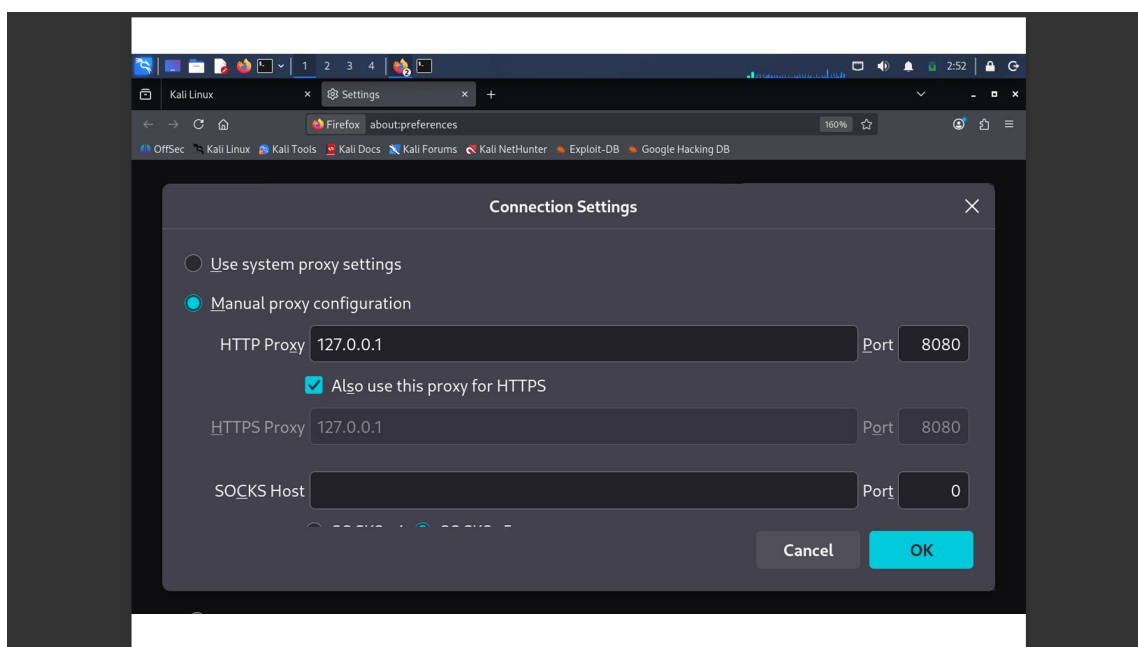


Figure 10: Firefox: manual proxy configured to 127.0.0.1:8080.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Interception Workflow

SECTION 08 | SECURITY ANALYSIS

The Proxy screenshots show interception enabled and a GET request to the local target appearing in Burp. This confirms that browser-to-target traffic is passing through the proxy. The request view exposes HTTP method, path, host, browser headers, accepted content types, and other metadata.

A disciplined interception workflow is: open the target, capture a request, inspect it without changing anything, forward it, verify the response, and then send a copy to Repeater for controlled modification. Interception should not remain enabled indefinitely because it can make ordinary browsing cumbersome. During testing, the tester can use Burp's scope and filtering features to limit interception to the intended host.

ASSESSMENT & SECURITY TAKEAWAY

A useful workflow is to capture a request, forward it unchanged, compare the result with browser behavior, and only then create a modified copy in Repeater. This prevents accidental confusion between a normal application response and a response caused by a test modification. Burp history also provides a timeline of requests, which can later be used to build the evidence trail for the final report.

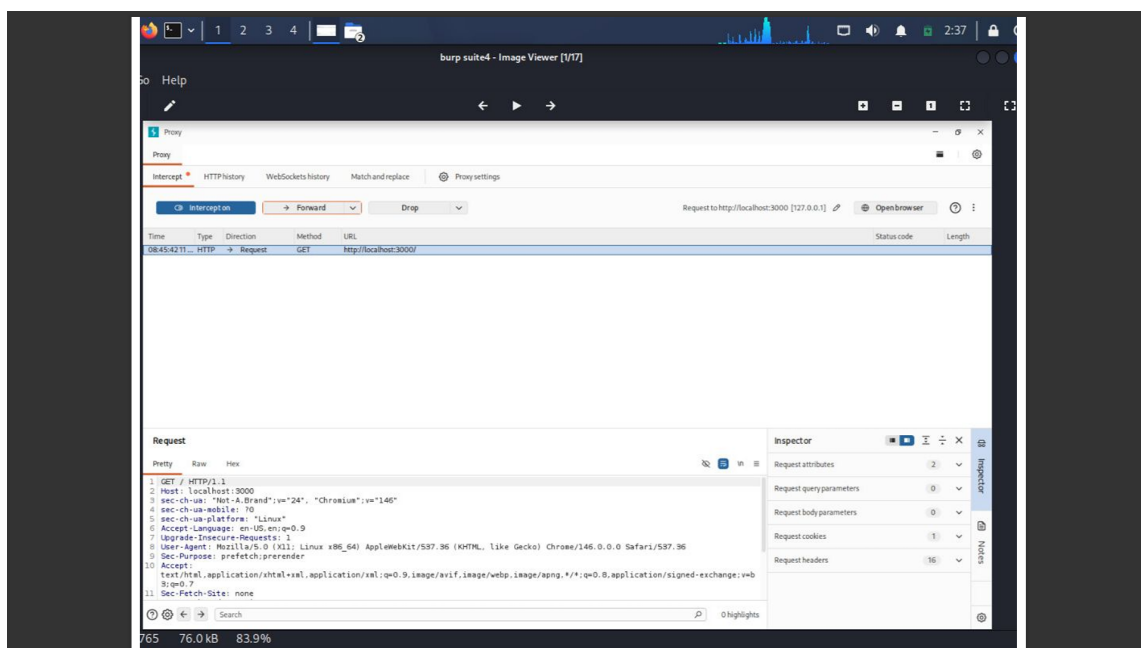


Figure 6: Burp Proxy: captured GET request to the local Juice Shop target.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Authentication Request Analysis

SECTION 09 | SECURITY ANALYSIS

The Repeater evidence contains a POST request to `/rest/user/login` with a JSON body containing an email and password. The response includes HTTP 200 OK and a JSON authentication object containing a token. This demonstrates a common token-based web API pattern: the client submits credentials, the server validates them, and a token is returned for subsequent authenticated requests.

Important fields to inspect include the HTTP method, endpoint, Content-Type, request body, response status, response content type, cookies, and authorization material. The presence of a token in the response does not by itself indicate a vulnerability. The security question is whether authentication and authorization are enforced correctly, whether tokens are protected, whether login attempts are rate limited, and whether tokens are accepted only in appropriate contexts.

ASSESSMENT & SECURITY TAKEAWAY

Authentication requests are high-value security boundaries because they process credentials and establish a session. Reviewers should examine whether credentials are sent over protected transport, whether error messages are consistent, whether authentication is rate limited, whether successful responses contain appropriately protected tokens, and whether later requests correctly require authentication. A secure design should also prevent authentication state from being confused with authorization to access specific objects.

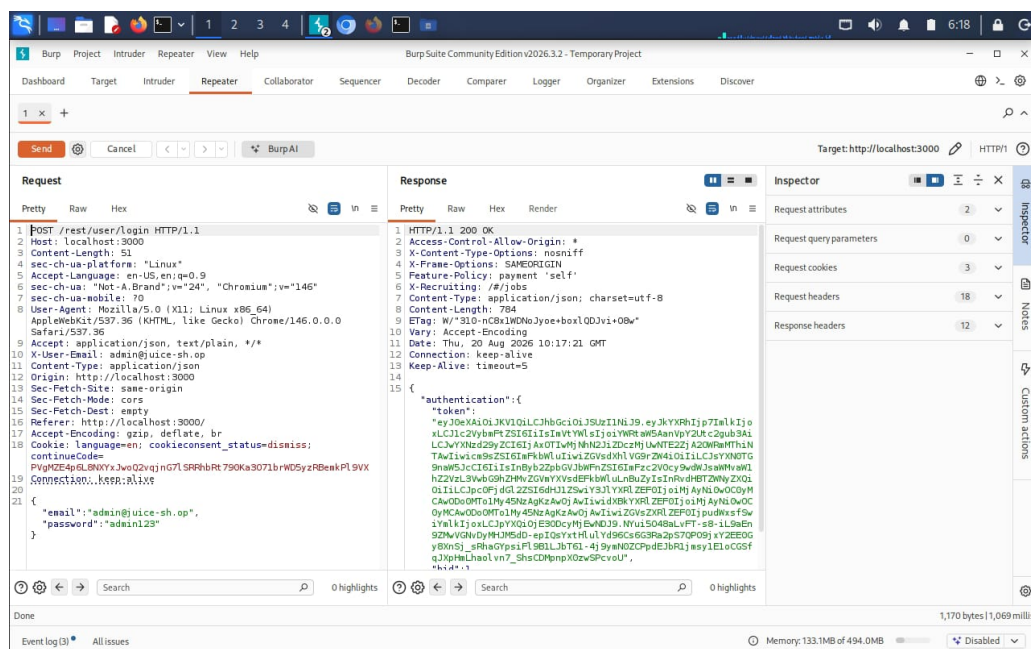


Figure 1: Burp Repeater: login request and successful JSON authentication response.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Credential Testing with Burp Intruder

SECTION 10 | SECURITY ANALYSIS

The Intruder screenshot shows a POST login request with a selected password parameter and a small simple-list payload set. The attack results show multiple attempts with different HTTP status codes, including several 401 Unauthorized responses and one 200 response. In a controlled lab, this pattern can indicate that one test credential produced a different application response and therefore deserves manual verification.

Intruder is useful for systematic testing, but it should be used carefully. A real application may lock accounts, trigger monitoring, or create operational impact. In this project the target is local and intentionally vulnerable. A proper assessment would define a very small payload list first, confirm the response-difference logic, and then stop once the test objective is met. Status code alone is not sufficient proof of a valid account; response body, length, headers, and application state should also be checked.

ASSESSMENT & SECURITY TAKEAWAY

Controlled credential testing should use a deliberately small dataset in a training environment. Response comparison should consider status code, response length, body text, headers, cookies, timing, and subsequent access rather than relying on one indicator. In production testing, automated attempts may trigger account lockouts or monitoring alerts, so written authorization and agreed rate limits are essential.

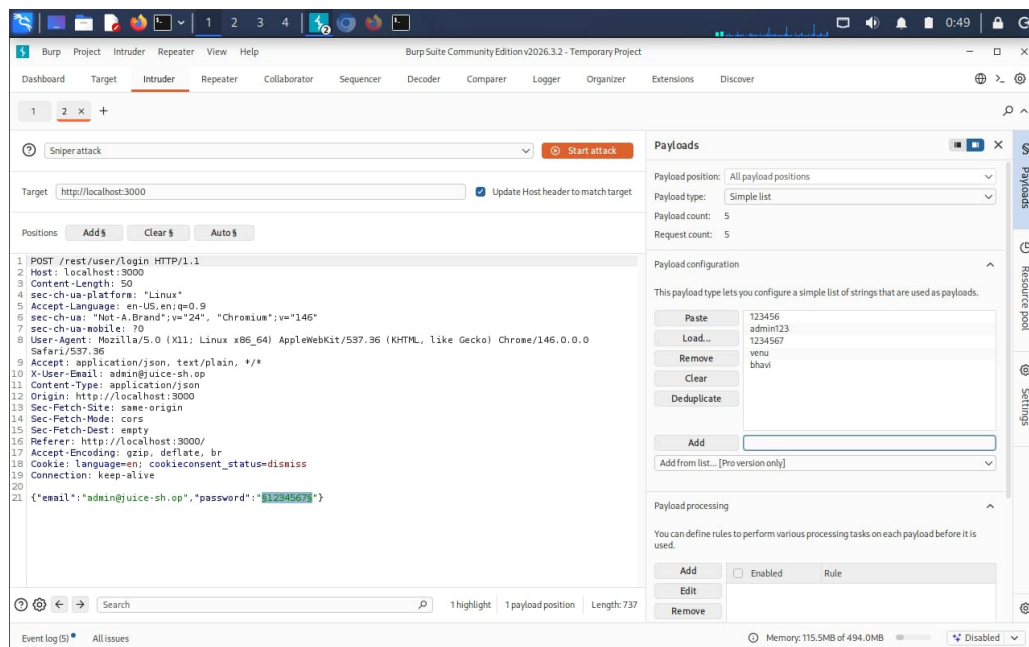


Figure 7: Burp Intruder: password payload position and a small test list.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Authentication Failure Analysis

SECTION 11 | SECURITY ANALYSIS

A separate Repeater screenshot shows a login attempt returning HTTP 401 Unauthorized with the message 'Invalid email or password.' This is a useful negative control. It establishes the expected failure behavior against which successful authentication responses can be compared.

Negative controls are essential in application testing because they help distinguish a meaningful change from a normal response. If a modified input changes the response from 401 to 200, the tester should investigate whether authentication succeeded, whether only the response format changed, or whether the application has inconsistent error handling. The evidence in this project shows both successful and unsuccessful authentication responses, providing a useful comparison set.

ASSESSMENT & SECURITY TAKEAWAY

Negative testing is as important as positive testing. A predictable 401 response establishes what the application looks like when authentication fails. A good assessment compares multiple invalid inputs and checks whether the application leaks different messages for unknown users versus incorrect passwords. Consistent errors reduce account enumeration risk. Rate limiting and monitoring should be applied regardless of whether the error message is generic.

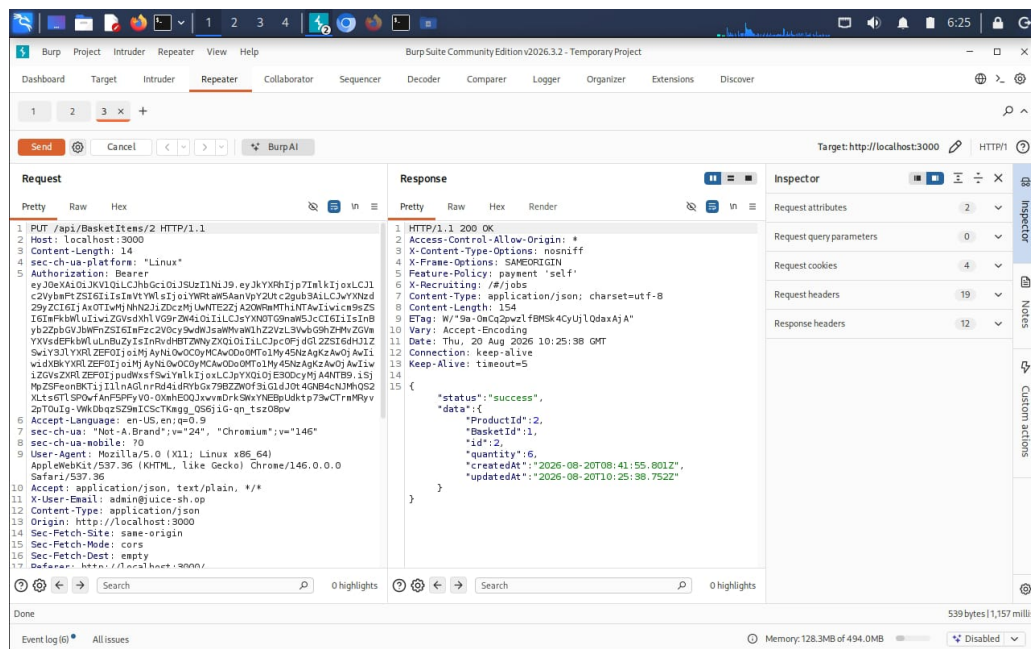


Figure 4: Burp Repeater: login request with captured headers and authentication response.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Authentication Input Manipulation

SECTION 12 | SECURITY ANALYSIS

One Repeater screenshot shows an authentication body containing a quote-based SQL-injection-style expression in the email field. The response is HTTP 200 and contains an authentication token. In the context of the intentionally vulnerable Juice Shop laboratory, this is strong evidence that the input manipulation changed the authentication outcome and should be investigated as a potential SQL injection/authentication-bypass finding.

The important lesson is that application code should never concatenate untrusted authentication input directly into SQL statements. Parameterized queries or prepared statements should be used. Server-side validation is still useful, but validation should complement rather than replace parameterization. Error messages should also avoid revealing database behavior, and authentication endpoints should have rate limiting and monitoring.

ASSESSMENT & SECURITY TAKEAWAY

Potential SQL injection or authentication bypass should be validated carefully. A complete investigation would compare the manipulated request with a normal failed login, review response differences, repeat the test with harmless variants, and, where authorized, inspect application or database logs. The preferred remediation is parameterized database access, supported by strict validation and safe error handling. The application should never construct SQL by concatenating untrusted authentication fields.

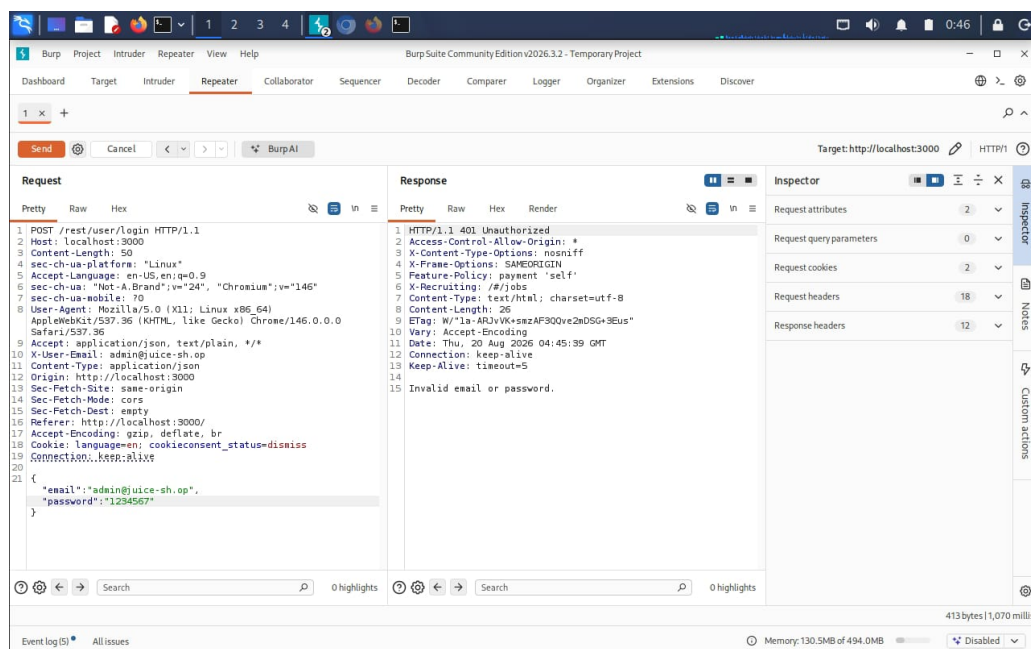


Figure 12: Burp Repeater: login request showing an alternate test input.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Token and Session Handling

SECTION 13 | SECURITY ANALYSIS

The successful login responses contain long token strings in JSON. These are consistent with JWT-style bearer tokens commonly used by APIs. The following authenticated requests include an Authorization: Bearer header, showing how the session context is propagated to protected API endpoints.

A security review should examine token lifetime, issuer and audience validation, signature verification, storage on the client, logout invalidation, privilege claims, and protection against token leakage. Tokens should never be treated as proof of authorization merely because they are syntactically valid. The server must validate the token and then enforce the requested operation against the authenticated user's privileges.

ASSESSMENT & SECURITY TAKEAWAY

Token security should be evaluated across the complete lifecycle: issuance, storage, transmission, validation, expiration, renewal, logout, and revocation. JWT-style tokens should have correctly validated signatures and claims such as issuer, audience, and expiry where applicable. Sensitive tokens should not appear in URLs or application logs. Reports shared outside the testing team should redact bearer credentials because possession of a valid token may provide access to protected functions.

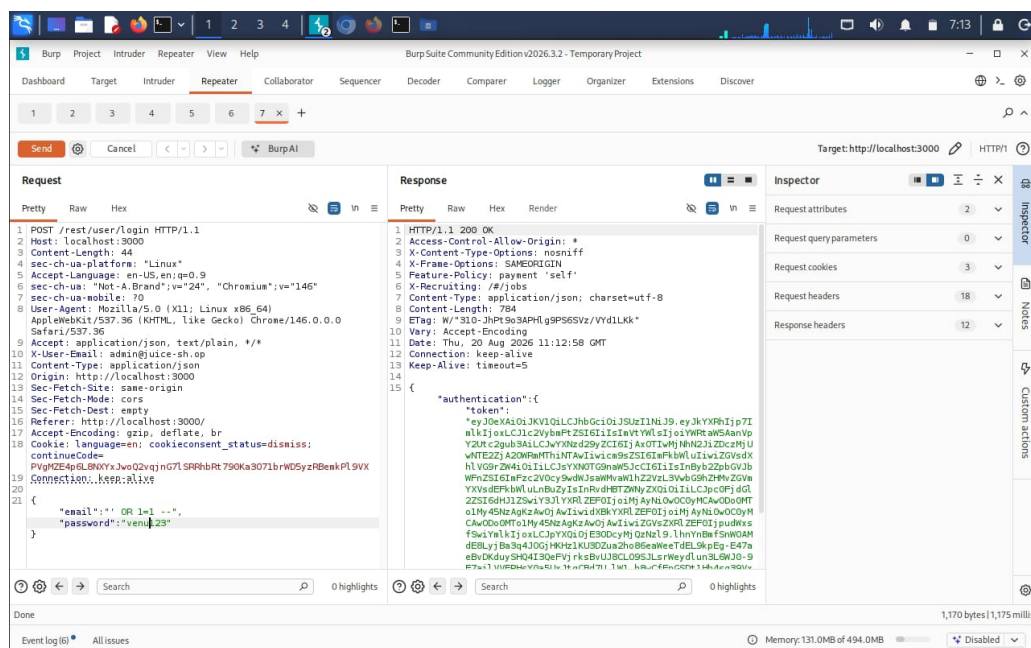


Figure 13: Burp Repeater: authenticated request/response with JWT-like token.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Basket API Authorization Testing

SECTION 14 | SECURITY ANALYSIS

The Repeater screenshots show an authenticated request to a basket endpoint such as GET /api/BasketItems/2 followed by a successful JSON response containing basket data. Another screenshot shows a PUT request to the same resource family and a successful response containing fields such as ProductId, BasketId, quantity, and timestamps.

This is an important authorization-testing area because object identifiers are visible in the URL. The correct security question is whether a logged-in user can access or modify only their own basket. Changing an object identifier and observing a successful response can indicate an insecure direct object reference or broken object-level authorization, but a confirmed finding requires demonstrating access to another user's object under controlled conditions. The report therefore treats these screenshots as evidence of authorization testing and API exposure, not as conclusive proof of an IDOR unless ownership was independently established.

ASSESSMENT & SECURITY TAKEAWAY

Object-level authorization is a common API security concern. A resource identifier in a URL should be treated as an untrusted reference, not proof that the caller owns the object. A robust test uses separate authorized test accounts and verifies whether changing an identifier permits cross-user read or modification. If cross-user access is confirmed, the impact can include unauthorized disclosure or modification of another user's data.

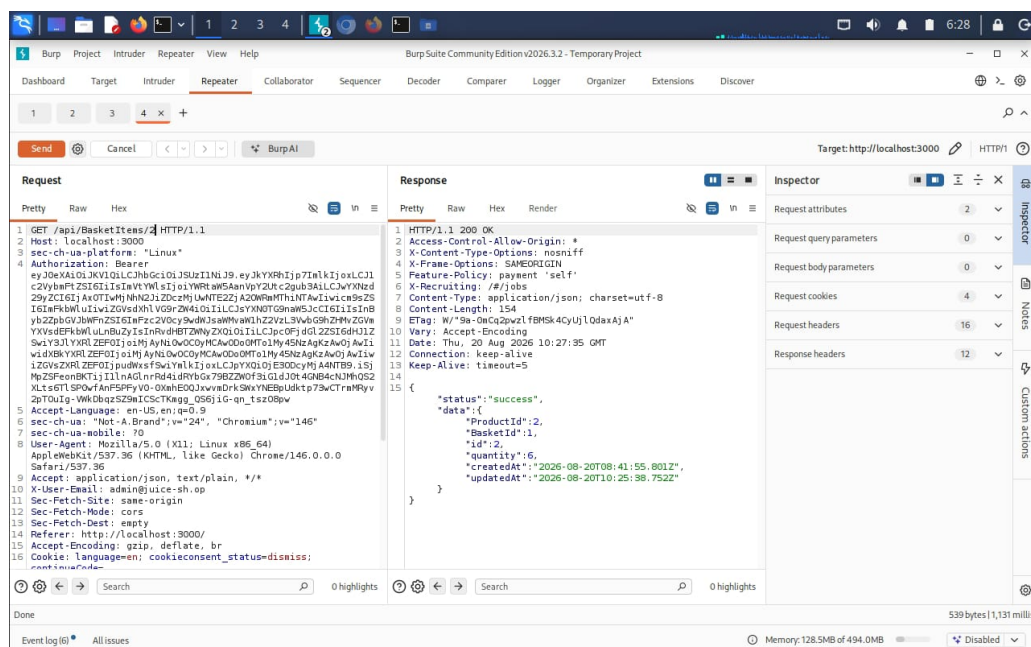


Figure 14: Burp Repeater: authenticated basket API request and successful response.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

HTTP Method and API Behavior

SECTION 15 | SECURITY ANALYSIS

The basket update evidence illustrates how the same API resource can support different HTTP methods. GET retrieves information while PUT changes server-side state. The successful response shows that the server processed the request and returned structured JSON.

Security testing should evaluate every supported method, not only GET. For state-changing operations, the tester should verify authentication, authorization, input validation, anti-CSRF requirements where cookie-based authentication is used, and business-rule enforcement. The server should not rely on hidden UI controls to prevent unauthorized actions. Any user capable of directly sending the HTTP request should still be subject to server-side authorization.

ASSESSMENT & SECURITY TAKEAWAY

Testing every supported HTTP method is important because security controls can be inconsistently implemented. GET, POST, PUT, PATCH, and DELETE should each be checked for authentication, authorization, validation, business rules, and safe error handling. State-changing methods deserve particular attention because successful manipulation may alter persistent application data.

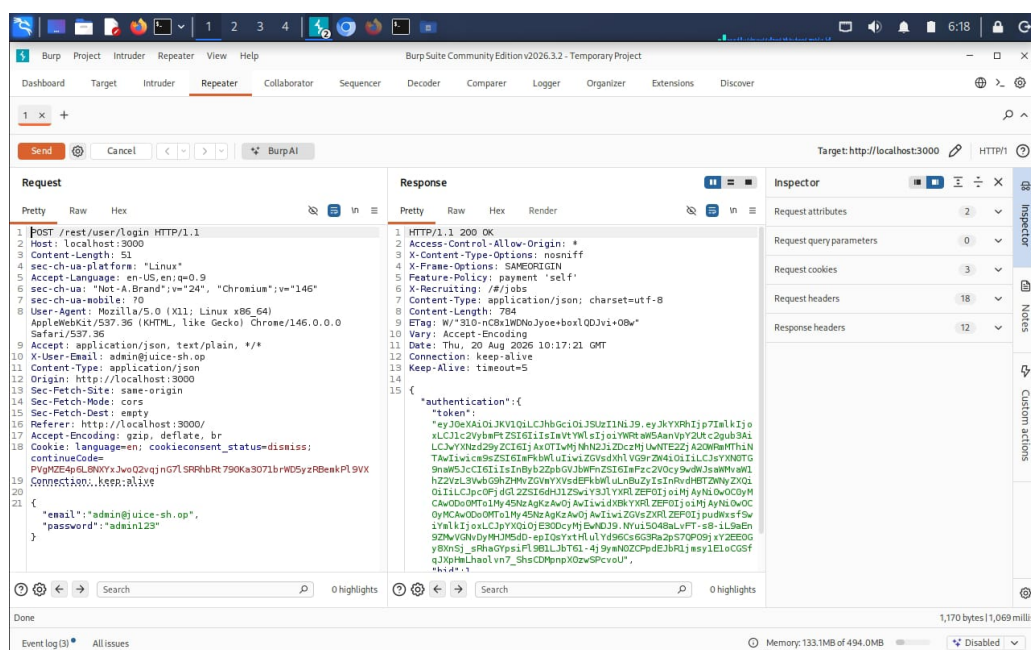


Figure 16: Burp Repeater: authenticated basket update request with successful response.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

User Endpoint and Data Exposure

SECTION 16 | SECURITY ANALYSIS

The screenshot showing GET /rest/user/whoami?fields=email demonstrates an API endpoint that returns information about the current user context. The response is JSON and the request is made without an obvious body. Such endpoints are common in modern single-page applications because the frontend needs identity information to render account-specific content.

The security assessment should consider whether the endpoint reveals only the minimum required fields, whether authentication is required, whether field selection can expose sensitive attributes, and whether authorization is correctly tied to the active session. A least-privilege design returns only the data needed by the client. Query parameters that control fields should be validated against an allow-list rather than allowing arbitrary database attributes to be exposed.

ASSESSMENT & SECURITY TAKEAWAY

Identity endpoints should follow data minimization. Returning an email address may be appropriate for a signed-in account page, while exposing internal IDs, roles, reset tokens, or other attributes may create unnecessary risk. Query parameters that select fields should use an allow-list. The endpoint should also behave predictably when no authenticated session exists.

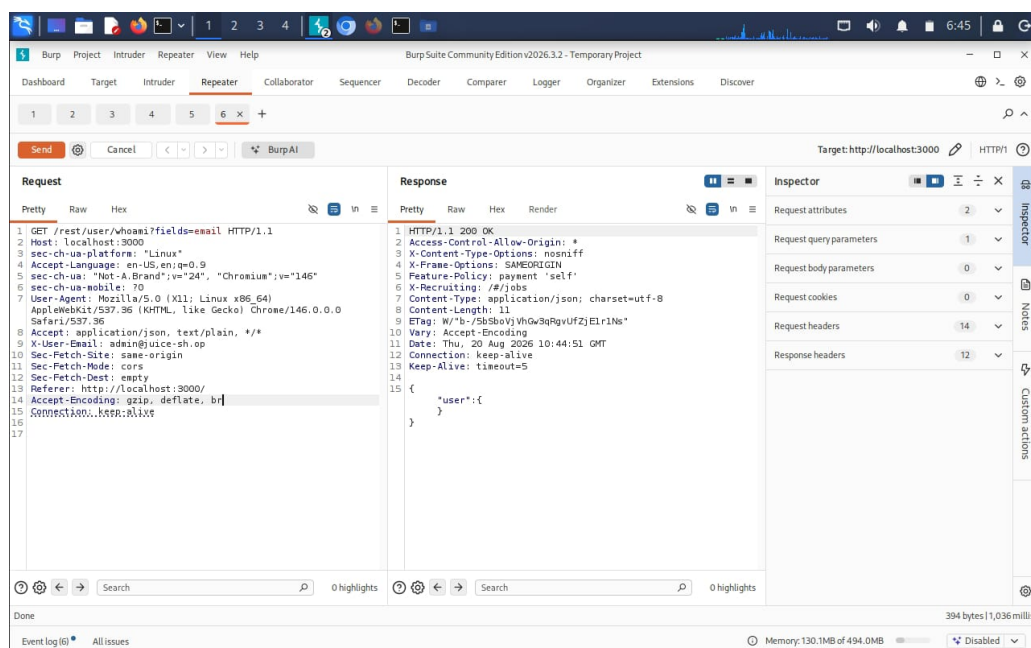


Figure 17: Burp Repeater: user endpoint request using the captured session context.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Cross-Site Scripting Test

SECTION 17 | SECURITY ANALYSIS

The final browser screenshot shows a search query containing an iframe/javascript-style XSS test string. This demonstrates an attempt to determine whether user-controlled search input is interpreted as executable markup rather than treated as plain text.

Cross-site scripting occurs when untrusted data reaches an HTML, JavaScript, or other executable context without appropriate encoding or sanitization. A secure application should render the search term as data. Depending on the context, defenses include context-aware output encoding, safe templating, input validation, a restrictive Content Security Policy, and avoiding dangerous DOM APIs. The correct test process is to start with a harmless proof-of-concept payload in the local lab and confirm whether script execution actually occurs. Merely displaying the string is not proof of XSS.

ASSESSMENT & SECURITY TAKEAWAY

XSS analysis should distinguish reflected, stored, and DOM-based behavior. A search parameter that is reflected into HTML is not automatically exploitable; the tester must determine the output context and whether the browser interprets the value as executable markup. Safe templating, contextual encoding, secure DOM APIs, and a restrictive Content Security Policy provide layered defenses.

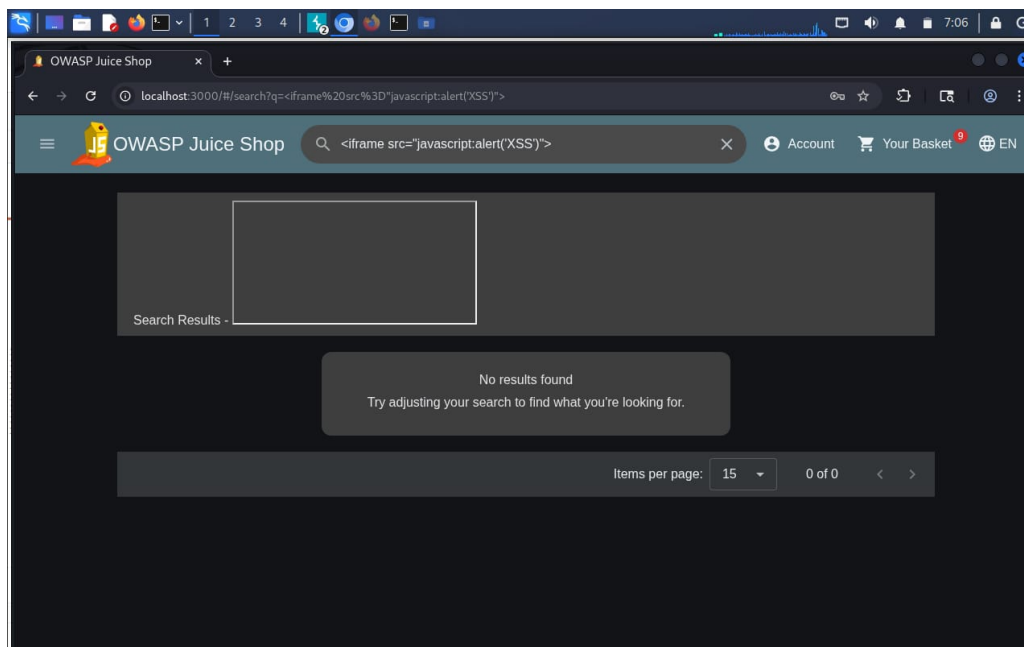


Figure 18: Juice Shop: search input containing a script/iframe-style XSS test string.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

CSRF Considerations

SECTION 18 | SECURITY ANALYSIS

The project evidence includes authentication and state-changing API requests, making cross-site request forgery an important security topic. CSRF is relevant when a browser automatically attaches ambient credentials, such as session cookies, to a cross-origin request. Bearer tokens kept outside automatically attached cookies can change the threat model, but state-changing operations still require careful authorization and origin validation.

Recommended controls include SameSite cookie attributes where cookies are used, anti-CSRF tokens for cookie-authenticated state changes, Origin/Referer validation where appropriate, and a strict separation between authentication and authorization. The screenshot evidence does not by itself prove a CSRF vulnerability; it establishes that state-changing requests exist and should be reviewed for CSRF protections.

ASSESSMENT & SECURITY TAKEAWAY

CSRF analysis begins by identifying how authentication credentials are attached to requests. Cookie-based sessions can be automatically included by browsers, making origin-based protections important for state-changing actions. SameSite cookie settings, anti-CSRF tokens, and Origin/Referer validation can reduce risk. Bearer-token APIs may have a different CSRF profile, but authorization and input validation remain necessary.

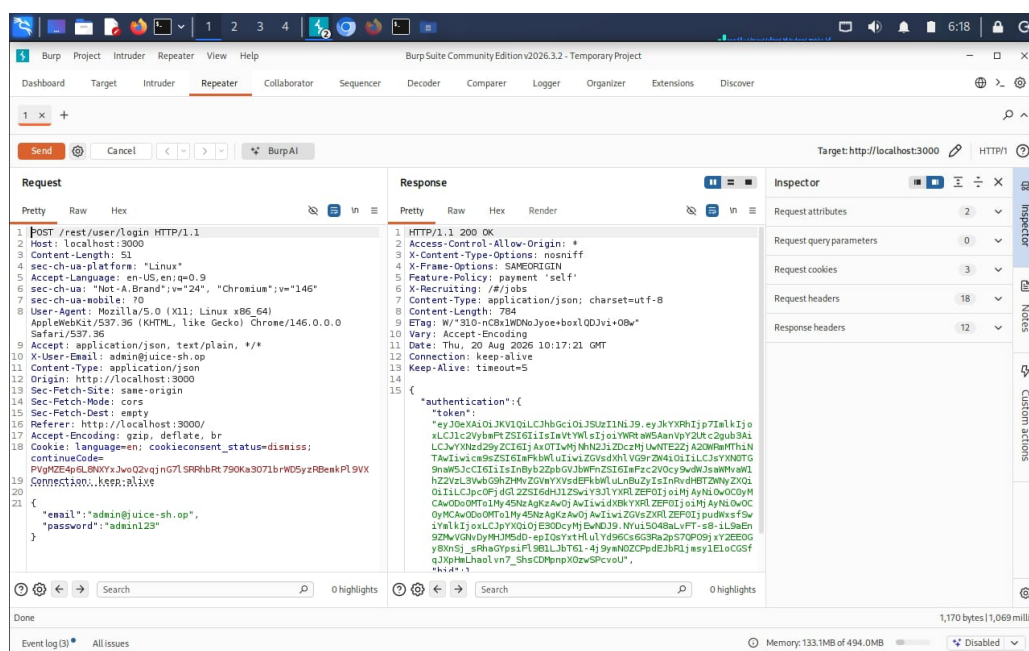


Figure 1: Burp Repeater: login request and successful JSON authentication response.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Findings and Risk Assessment

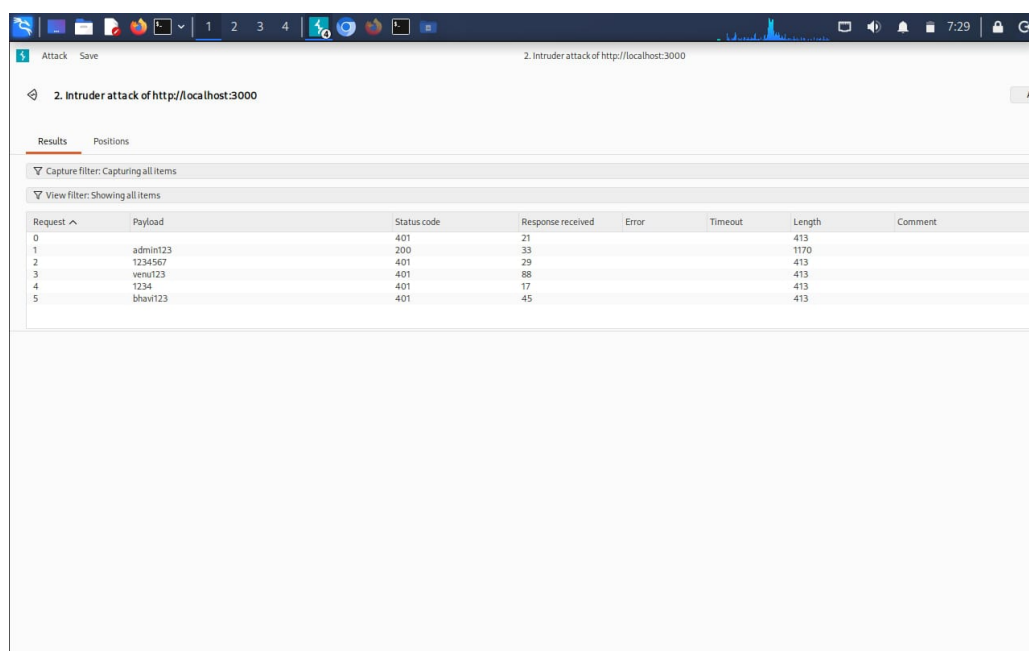
SECTION 19 | SECURITY ANALYSIS

Based on the screenshots, the most significant areas for further investigation are authentication input handling, automated credential testing resistance, object-level API authorization, token/session security, data minimization, and XSS protection. The quote-based login test producing a successful-looking authentication response is the strongest evidence of a potentially exploitable authentication/SQL-injection issue in this lab.

The Intruder results also indicate that the login endpoint produces distinguishable responses for different credentials. Even if a password test is not itself a vulnerability, a production system should apply rate limiting, progressive delays, account protection, monitoring, and strong password policies. Basket endpoints should enforce ownership server-side, and user endpoints should return only necessary data. Search input should be safely encoded for its output context.

ASSESSMENT & SECURITY TAKEAWAY

Risk should be assigned using impact, exploitability, required privileges, attack complexity, and confidence in the evidence. A potentially exploitable authentication flaw generally receives higher priority than an informational header observation because it can affect account access. API authorization weaknesses may also be high impact when they permit cross-user data access or modification.



The screenshot shows the Burp Intruder interface with an attack titled "2. Intruder attack of http://localhost:3000". The "Results" tab is active, displaying a table of captured items. The table has columns for Request, Payload, Status code, Response received, Error, Timeout, Length, and Comment. Five requests are listed, each with a different payload and a status code of 401.

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		401	21			413	
1	admin123	200	33			1170	
2	1234567	401	29			413	
3	venu123	401	88			413	
4	1234	401	17			413	
5	bhavi123	401	45			413	

Figure 8: Burp Intruder: attack results showing different HTTP status codes.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Remediation Recommendations

SECTION 20 | SECURITY ANALYSIS

1. Use parameterized SQL queries or ORM parameter binding for all database operations.
2. Implement robust server-side authentication and authorization checks on every protected endpoint.
3. Enforce object-level authorization using the authenticated user's identity and ownership relationship rather than trusting URL IDs.
4. Add rate limiting, lockout or adaptive throttling, and monitoring to authentication endpoints.
5. Protect session tokens with appropriate lifetime, validation, storage, and revocation controls.
6. Encode untrusted data according to output context and avoid unsafe DOM insertion APIs.
7. Apply a restrictive Content Security Policy as defense in depth.
8. Use SameSite cookies and anti-CSRF protections where cookie-based authentication is used.
9. Return only required user fields and validate field-selection parameters.
10. Log security-relevant events while avoiding passwords, bearer tokens, and other secrets in logs.

ASSESSMENT & SECURITY TAKEAWAY

Remediation should be prioritized rather than treated as a flat list. First address authentication bypass or injection risks, then enforce object-level authorization, then strengthen session/token controls and abuse prevention, and finally apply defense-in-depth controls such as CSP, secure headers, and improved logging. After fixes are implemented, each original test should be repeated to confirm the behavior has changed without breaking legitimate functionality.

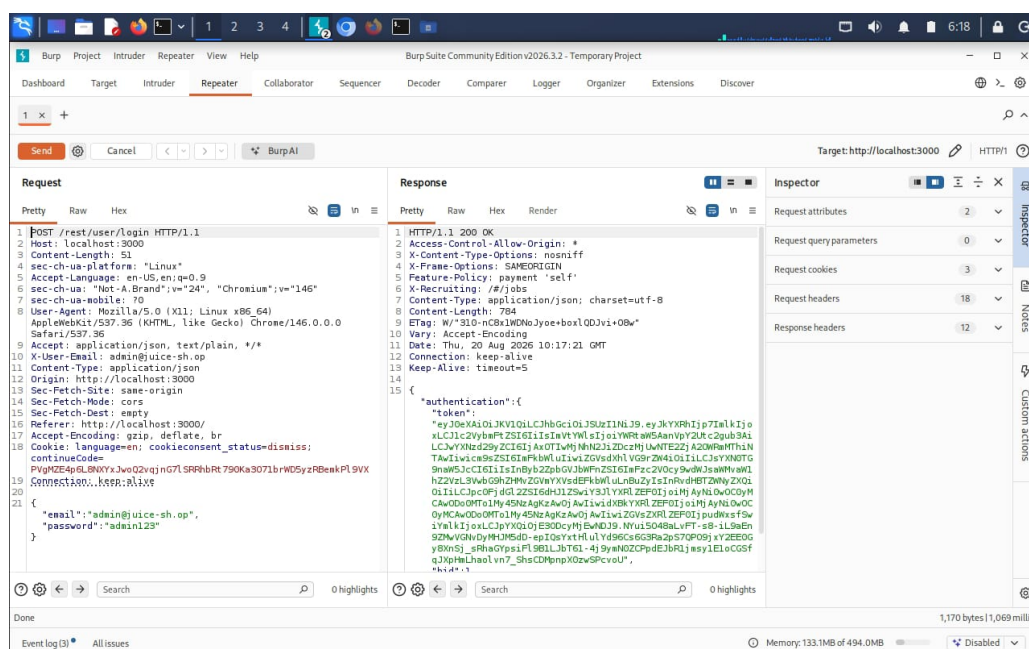


Figure 1: Burp Repeater: login request and successful JSON authentication response.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Test Methodology and Reproducibility

SECTION 21 | SECURITY ANALYSIS

The assessment follows a repeatable sequence: deploy the target, verify availability, configure the browser proxy, capture baseline traffic, identify security-sensitive endpoints, send requests to Repeater, modify one parameter at a time, compare responses, and document evidence. Intruder is used only for a small controlled credential test set. The browser is then used to demonstrate a user-input security test.

For reproducibility, a future tester should record Docker image/tag, container port mapping, browser proxy settings, Burp project state, exact request paths, test data, and the expected baseline responses. Each finding should include a description, affected endpoint, precondition, evidence, impact, severity, and remediation. This structure turns a collection of screenshots into a professional security assessment.

ASSESSMENT & SECURITY TAKEAWAY

A strong evidence package contains the test objective, target endpoint, precondition, exact request change, expected result, observed result, and screenshot reference. Keeping these elements together makes the report useful for developers and auditors. Retesting should use the same baseline request and the same controlled test account so remediation can be compared objectively.

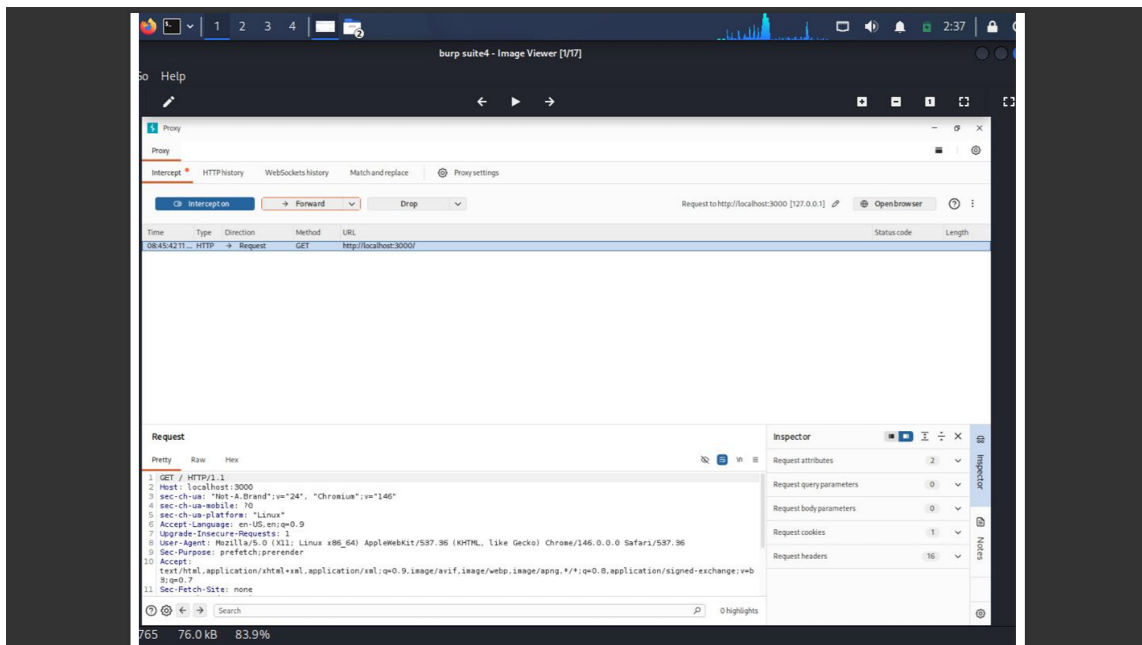


Figure 6: Burp Proxy: captured GET request to the local Juice Shop target.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Limitations

SECTION 22 | SECURITY ANALYSIS

The report is based on the supplied screenshots rather than a live re-execution of the environment. Therefore, conclusions are limited to what can be observed in the evidence. A screenshot can show that a request was sent and a response was received, but it may not establish ownership of an object, persistence of a session, exact application version, or the full server-side cause of a response.

The project also does not provide source code, server logs, database records, deployment configuration files, or a complete list of endpoints. Consequently, the report avoids claiming vulnerabilities that require evidence not visible in the screenshots. A full professional assessment would combine dynamic testing with source review, configuration review, dependency analysis, and server-side telemetry.

ASSESSMENT & SECURITY TAKEAWAY

The absence of source code, server logs, database state, and complete endpoint documentation limits the certainty of some conclusions. For example, a successful API response does not by itself prove that a resource belonged to another user. The report therefore uses confidence-aware language and identifies where additional verification would be required.

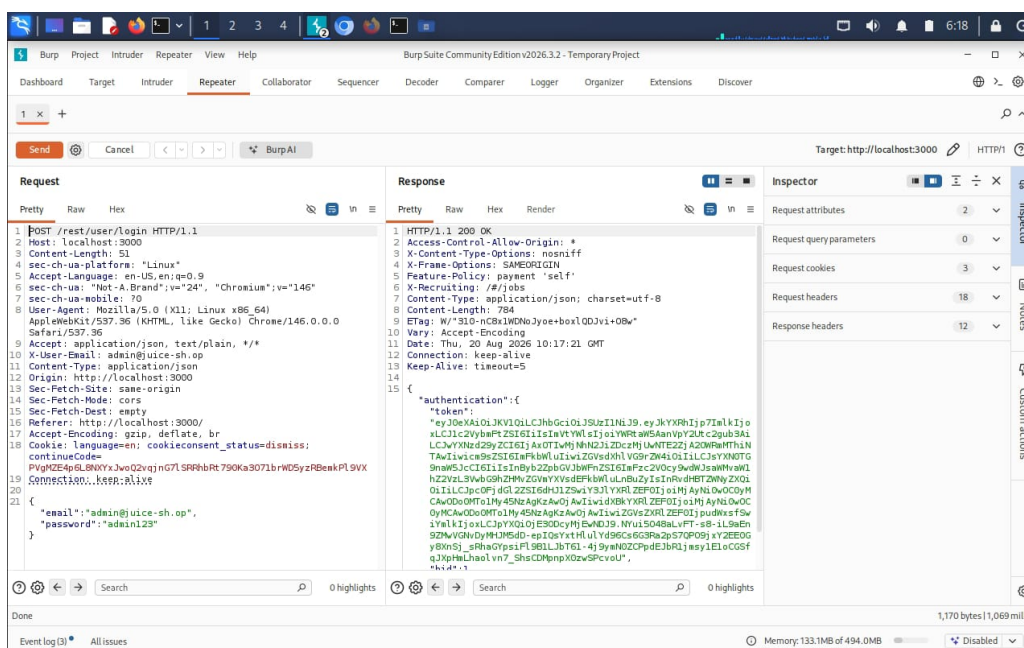


Figure 1: Burp Repeater: login request and successful JSON authentication response.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Conclusion

SECTION 23 | SECURITY ANALYSIS

This project demonstrates a complete introductory web application security testing workflow against an intentionally vulnerable OWASP Juice Shop instance. Docker provided a controlled target, Kali Linux provided the security-testing environment, Firefox acted as the client, and Burp Suite provided visibility and manipulation of HTTP traffic. The screenshots document proxy configuration, request interception, authentication analysis, Intruder-based credential testing, authenticated API testing, user endpoint inspection, and XSS-oriented input testing.

The central security lesson is that application security must be enforced at the server and API layers. Client-side controls and UI restrictions are not sufficient. Authentication must be robust, authorization must be checked for every object and operation, untrusted input must be safely handled, and security-sensitive endpoints must resist automation and abuse. The laboratory provides a safe environment for learning these concepts before applying them to authorized real-world assessments.

ASSESSMENT & SECURITY TAKEAWAY

The practical value of the project is the connection between theory and raw HTTP evidence. Instead of discussing vulnerabilities only conceptually, the workflow shows how a tester observes real requests, modifies them, compares responses, and derives security questions from application behavior. This provides a strong foundation for more advanced authorized testing such as API authorization matrices, business-logic testing, dependency analysis, and secure-code review.

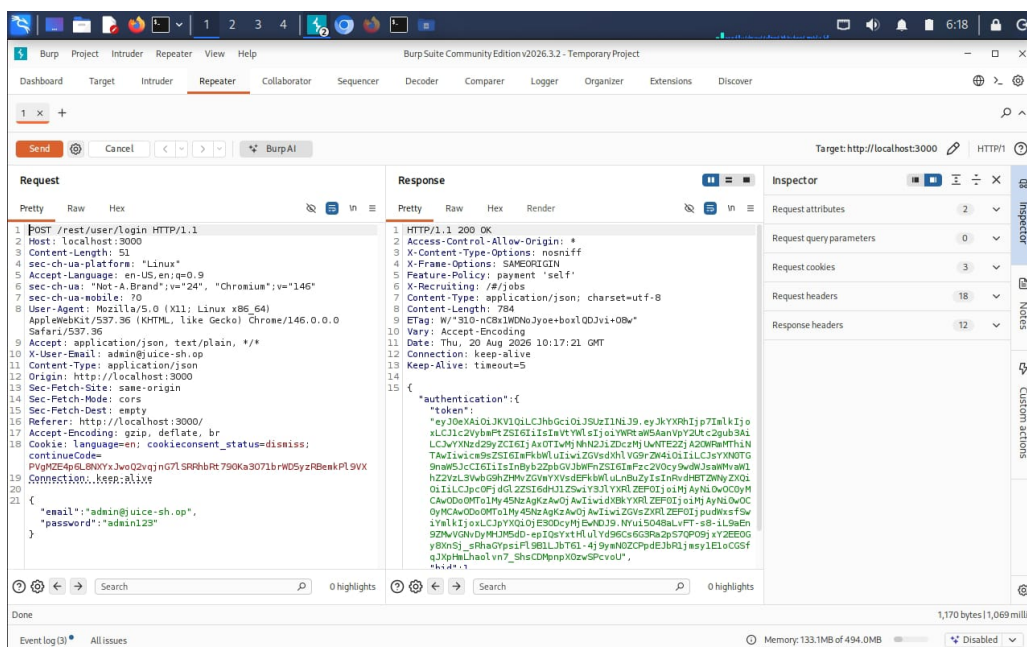


Figure 1: Burp Repeater: login request and successful JSON authentication response.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Appendix A – Evidence Index

SECTION 24 | SECURITY ANALYSIS

- Figure 1: Burp login request and authentication response.
- Figure 2: Docker image pull.
- Figure 3: Docker container running with port 3000 mapped.
- Figure 4: Repeater authentication traffic.
- Figure 5: Burp Proxy interception screen.
- Figure 6: Captured GET request to localhost:3000.
- Figure 7: Intruder payload configuration.
- Figure 8: Intruder result table.
- Figure 9: Juice Shop browser interface.
- Figure 10: Firefox manual proxy settings.
- Figure 11: Successful authentication response.
- Figure 12: Authentication input manipulation.
- Figure 13: Token-bearing authenticated response.
- Figure 14: Basket GET API response.
- Figure 15: Basket API request/response.
- Figure 16: Basket PUT update response.
- Figure 17: User 'whoami' endpoint.
- Figure 18: XSS-oriented search input.

These figures are reproduced from the screenshots supplied for the project and are used as visual evidence for the corresponding sections.

ASSESSMENT & SECURITY TAKEAWAY

Evidence handling note: each screenshot should be retained with its original filename and associated test objective. When a report is shared externally, secrets such as passwords, session cookies, authorization tokens, or personally identifying information should be redacted. The screenshots in this project are included to demonstrate methodology and results within the controlled training environment.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Appendix B – Security Testing Checklist

SECTION 25 | SECURITY ANALYSIS

Before testing: confirm authorization, define scope, verify the local target, and create a clean test account. During testing: capture baseline requests, inspect authentication, test authorization for each object, test state-changing methods, review input handling, check session behavior, and record response differences. After testing: stop intrusive automation, remove test data where appropriate, invalidate test sessions, preserve evidence, classify findings, and document remediation.

For future work, the project can be extended with endpoint discovery, access-control matrix testing, API schema review, dependency vulnerability scanning, security-header review, password-policy testing, file-upload validation, business-logic testing, and authenticated/unauthenticated comparison matrices.

ASSESSMENT & SECURITY TAKEAWAY

Completion criterion: a test should be considered complete only when the request has been captured, the expected baseline is known, the security control has been exercised, the response has been evaluated, and the evidence has been recorded. Findings should then be retested after remediation.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Appendix C – Final Project Summary

SECTION 26 | SECURITY ANALYSIS

The completed project demonstrates an end-to-end web application security assessment performed in a controlled local laboratory environment. The work begins with deployment of OWASP Juice Shop using Docker and verification that the application is accessible through the expected localhost port. This initial stage establishes a known and repeatable baseline before any security testing is performed.

The next stage focuses on traffic visibility and request analysis. Firefox is configured to route traffic through Burp Suite, allowing browser requests to be captured and examined at the HTTP level. This makes it possible to identify request methods, paths, headers, cookies, authentication data, parameters, response codes, and response bodies. Burp Repeater is then used to resend selected requests in a controlled manner, making it easier to compare normal behavior with modified requests.

Authentication testing forms an important part of the assessment. The report examines successful and unsuccessful login behavior, request manipulation, controlled Intruder testing, and the handling of authentication responses. The testing highlights why applications should use consistent error handling, rate limiting, secure credential processing, and strong server-side authentication controls. The presence of an authentication token is also considered from a session-security perspective, including the importance of protecting tokens during transmission and preventing unnecessary exposure in logs or reports.

API security is another major component of the project. Authenticated requests to basket and user-related endpoints demonstrate how modern web applications rely heavily on APIs for application functionality. The assessment therefore considers object-level authorization, HTTP methods, resource identifiers, input validation, and the possibility of unauthorized access when server-side authorization checks are incomplete. A key professional lesson is that authentication proves who a user is, while authorization determines what that user is actually allowed to access or modify.

The project also introduces input-validation and client-side attack considerations through controlled XSS testing. The presence of a suspicious string in a request is not treated as automatic proof of vulnerability; instead, the tester must determine how the application processes and renders the value. This distinction between an attack indicator and a confirmed vulnerability is essential for accurate security reporting. Similar care is applied to CSRF and session-management considerations, where the final security conclusion depends on the application's authentication architecture and server-side controls.

From a reporting perspective, the project demonstrates the importance of evidence-based conclusions. Each significant test is connected to an observable request, response, screenshot, or application behavior. The exact evidence filename is included with the corresponding figure so that a reviewer can trace each observation back to the original image. This improves auditability, makes the report easier to review, and supports future retesting after remediation.

The overall security assessment follows a repeatable lifecycle: define the authorized scope, deploy the target, establish normal behavior, capture traffic, analyze authentication and authorization, test controlled inputs, document evidence, assess risk, recommend remediation, and retest. This methodology can be extended to future authorized assessments involving API security testing, business-logic validation, access-control matrices, dependency review, secure configuration analysis, and source-code review.

Finally, the project provides practical experience in thinking like both an attacker and a defender. The tester learns how small changes to an HTTP request can reveal important security questions, while the defensive perspective focuses on validating every request on the server, minimizing exposed data, protecting authentication material, enforcing authorization consistently, and maintaining useful security logs. The combination of technical testing, evidence collection, risk assessment, and remediation planning makes the project a strong practical demonstration of foundational web application security skills.

REVIEWER FOCUS

Confirm the observed behavior against the application baseline, avoid treating a single response code as proof of exploitation, and preserve the request/response evidence needed for retesting.

Evidence Filename Index

These are the exact image filenames used in the report and referenced for evidence traceability.

FIGURE	EXACT IMAGE FILENAME
Figure 1	CSRF(1).jpeg
Figure 2	docker(1).png
Figure 3	docker1(1).png
Figure 4	9d17ee16-26c3-4ae5-b7bb-f80883d9e019.jpeg
Figure 5	52297393-b46c-4935-a3a6-39e91756e23a.png
Figure 6	25065999-a077-401c-9038-975ac1a664d9.png
Figure 7	d53f1e2d-1053-4dd2-896e-a106c871ecb0.jpeg
Figure 8	478d1b20-f546-4c69-8323-c58e39af9b9b.jpeg
Figure 9	2ef6e90d-100b-4f34-8403-ae03630cbcc5.jpeg
Figure 10	9b13a3f7-628c-4b55-8683-135c8b1a2669.png
Figure 11	a6685c1f-b751-44c1-b73c-68fb0103d8c1.jpeg
Figure 12	717c0982-2086-4950-8903-fa5324b61719.jpeg
Figure 13	3a499764-688f-4ef4-8569-3fceb5602b01.jpeg
Figure 14	298bf5d6-1886-4cc8-86d0-6e58d17a6cf1.jpeg
Figure 15	28181c85-ecef-4ce9-9766-39b63cba8ac1.jpeg
Figure 16	7add1613-b3a1-455f-ba22-a18925c49f27.jpeg
Figure 17	77c1aa8e-dcf4-4784-bb4b-e01a2ef6c0f4.jpeg
Figure 18	75a1546b-99e0-4ee0-98a4-bd7069057c91.jpeg